





NatureProtector

**Projeto e Seminário
Apresentação de Progresso**

**ISEL
LEIC**

**Alunos:
Miguel Alves
Gabriel Mano**



2025/2026

**Orientadores:
Artur Ferreira
Nuno Leite**

A proteger florestas para gerações futuras

Ideia reconhecida com o 3º lugar num concurso de startups

Organização da apresentação

- 1. Problema e motivação**
- 2. Módulo de prevenção**
- 3. Pesquisa e cenários**
- 4. Requisitos abordados**
- 5. Pipeline e arquitetura**
- 6. Trabalho futuro**
- 7. Demonstração**
- 8. Slides extras para questões**

Problema na atual gestão de incêndios

Os incêndios florestais tornaram-se um risco estrutural, mais frequente, imprevisível e difícil de gerir.

Métodos atuais

- Vigilância humana
- Câmaras
- Satélites / dados remotos

Problemas:

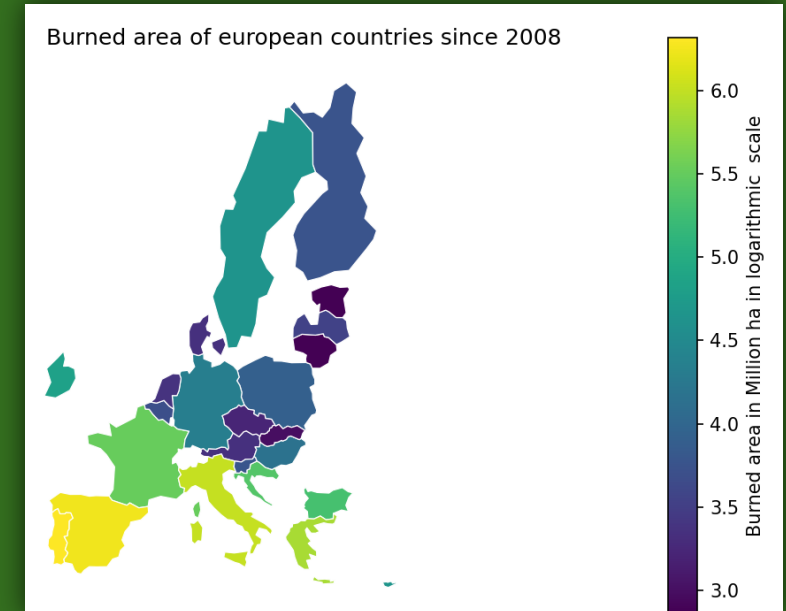
- Cobertura não homogênea
- Pontos cegos
- Detecção tardia
- Dependência humana

Lacunas

- Risco crescente
- Resposta reativa



<https://www.esquerda.net/artigo/incendio-de-pedrogao-grande-alastra-distritos-vizinhos/49289>



Países do Sul Europeu



Desde 2008, ~1/4 ardeu

Dados originados de Copernicus/EFFIS
Gráficos feitos por Miguel

Nature Protector, módulo de prevenção

Tecnologias

- .NET / C#
- PostgreSQL
- InfluxDB
- RabbitMQ
- Grafana
- Docker Compose

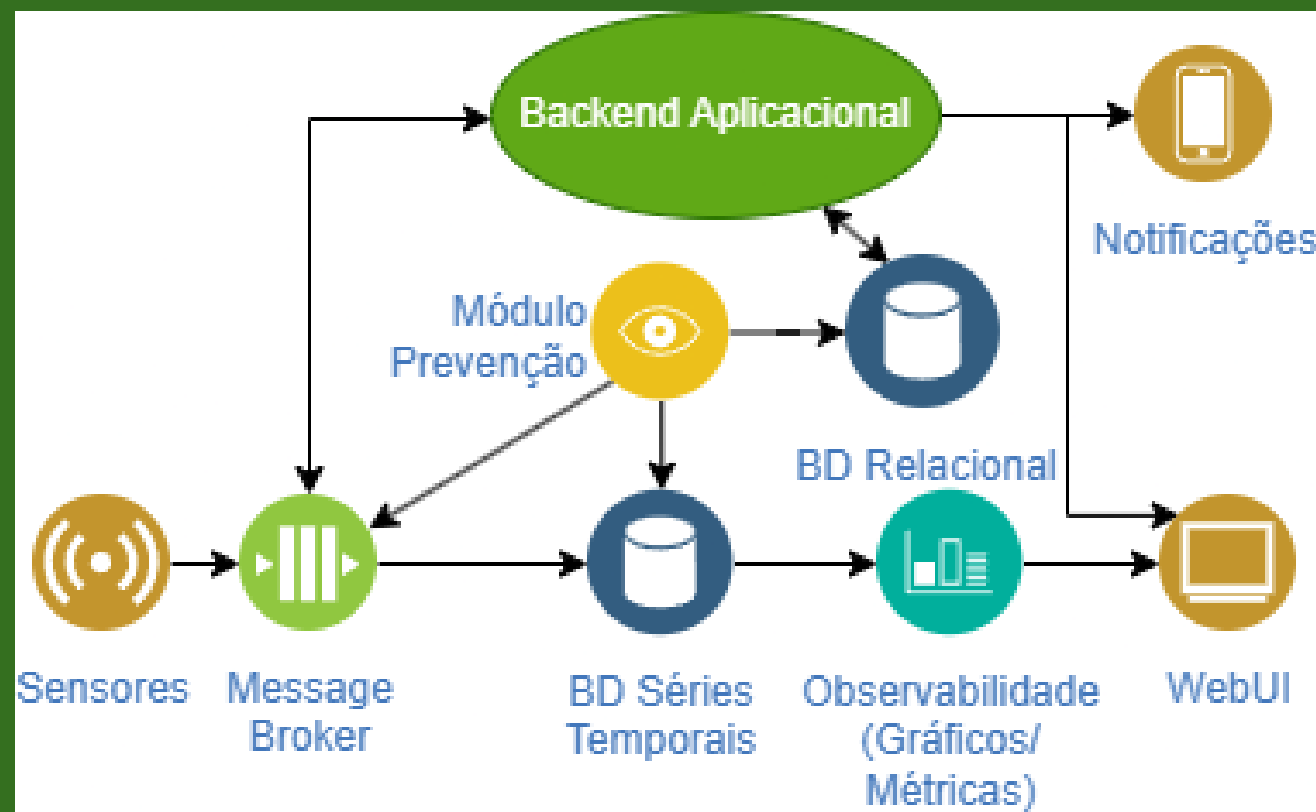
Capacidades atuais

- Simulação de sensores e cenários
- Transporte por RabbitMQ
- Persistência em PostgreSQL e InfluxDB
- Avaliação de risco preventivo inicial
- Observabilidade em Grafana

Evolução prevista

- Simulação mais fiável
- Utilização de índices reais
- Escala superior e maior distribuição

O Nature Protector é uma solução **preventiva**, **modular**, **rastreável** e **extensível**, concebida para demonstração credível no presente e evolução estruturada no futuro.



Pesquisa: área piloto, dados e cenários

Área piloto: Proença-a-Nova

Critérios de escolha de área

- Histórico de incêndios
- Cobertura florestal
- Clima relevante para o risco
- Disponibilidade de dados

Fontes de dados

- IPMA
- ICNF
- ERA5 / ERA5-Land
- CEMS / EFFIS
- FIRMS
- CORINE / TCD

Variáveis

- Temperatura
- Humidade relativa
- Vento
- Precipitação, como contexto
- Cobertura do solo / vegetação
- Histórico de fogos



Cenário A Dia normal

- Condições plausíveis
- Risco baixo / moderado

Cenário B Dia crítico

- Calor
- Baixa humidade
- Vento mais forte
- Maior risco

Cenário C Dia com perturbações

- Ruído
- Falhas
- Duplicados e atrasos
- Leituras suspeitas

Requisitos e como foram abordados

Funcionais

Configurar áreas e grelhas de risco

- Proença-a-Nova, grelha 1 km × 1 km, bootstrapping

Executar cenários previamente preparados

- Cenários A, B e C

Gerar leituras ambientais plausíveis

- Uso de dados reais como base

Cálculo e risco

- Cálculo simbólico com preparação para índices reais

Disponibilizar visualização

- Dashboards e UI

Não Funcionais

Reprodutibilidade

- Utilização de seed na geração de números aleatórios

Rastreabilidade

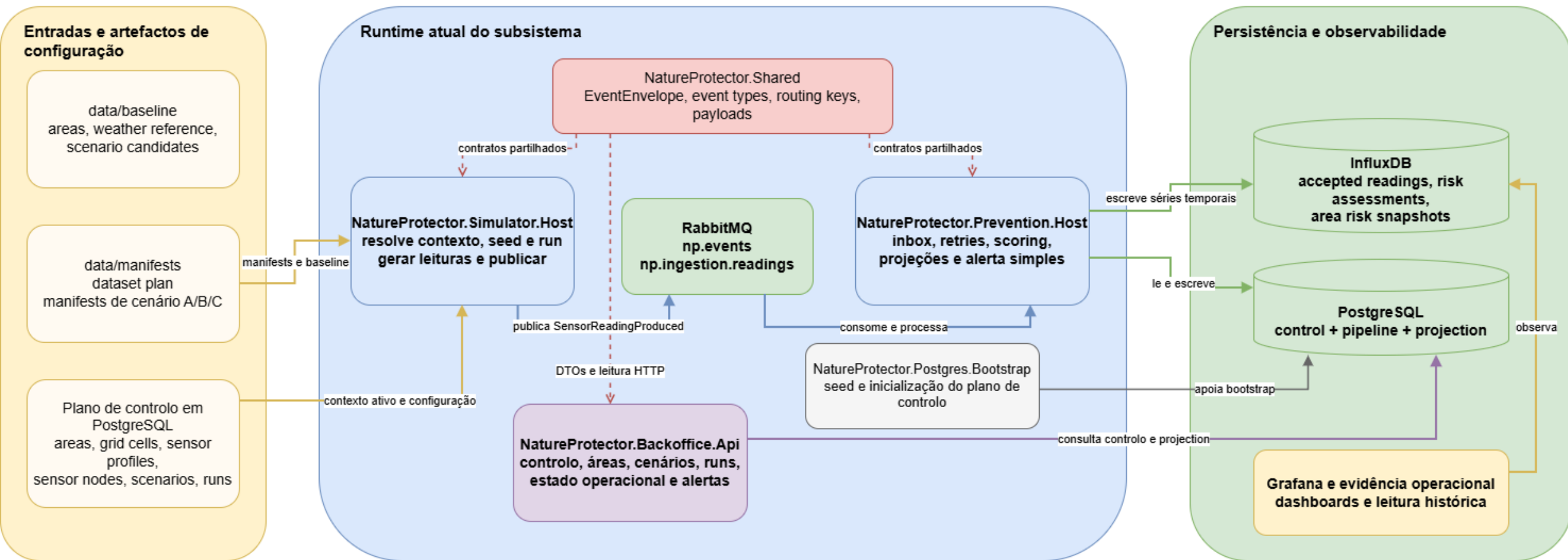
- Uso de correlation_id

Observabilidade

- Logs, filas, métricas e estado

Paisagem lógica do subsistema de prevenção

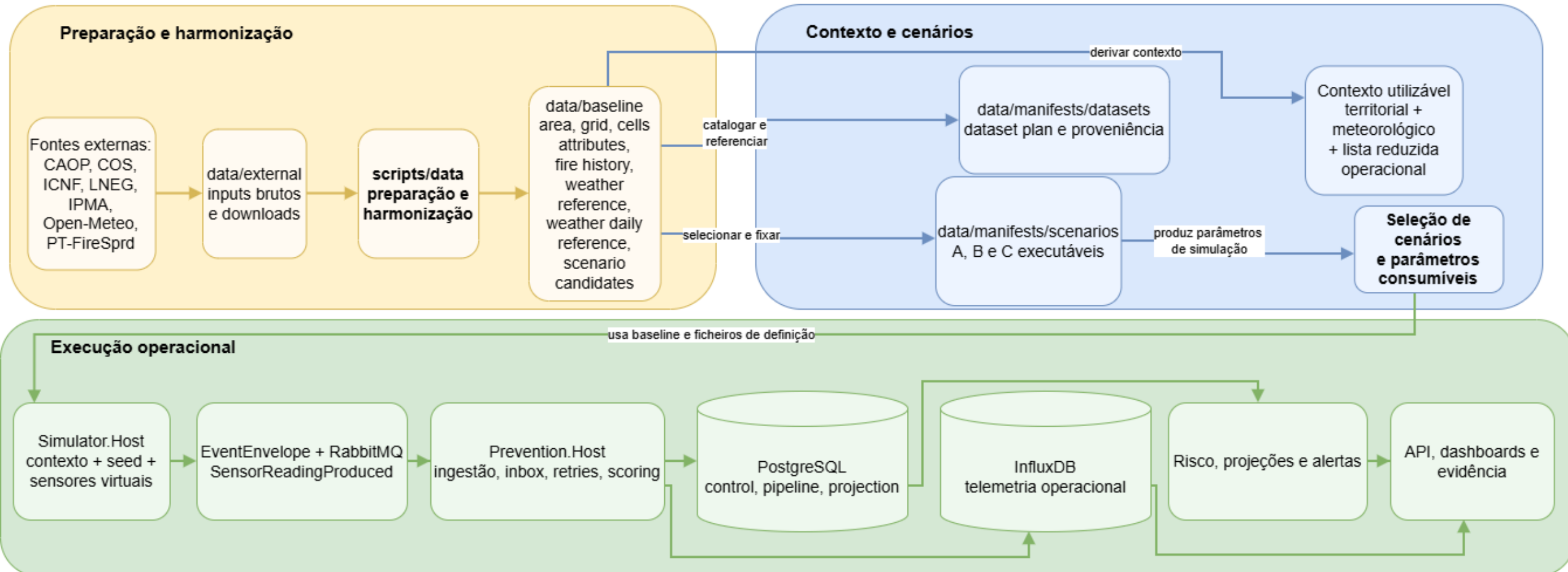
Vista de alto nível da runtime atual: o simulador publica leituras, a prevenção consome e persiste, e a API lê o plano de controlo e o estado operacional.



Aqui mostramos a visão de alto nível:
Configuração e datasets entram no sistema, o runtime executa a simulação e a prevenção, e os resultados ficam disponíveis em persistência e na observabilidade.

Cadeia end-to-end de dados do NatureProtector

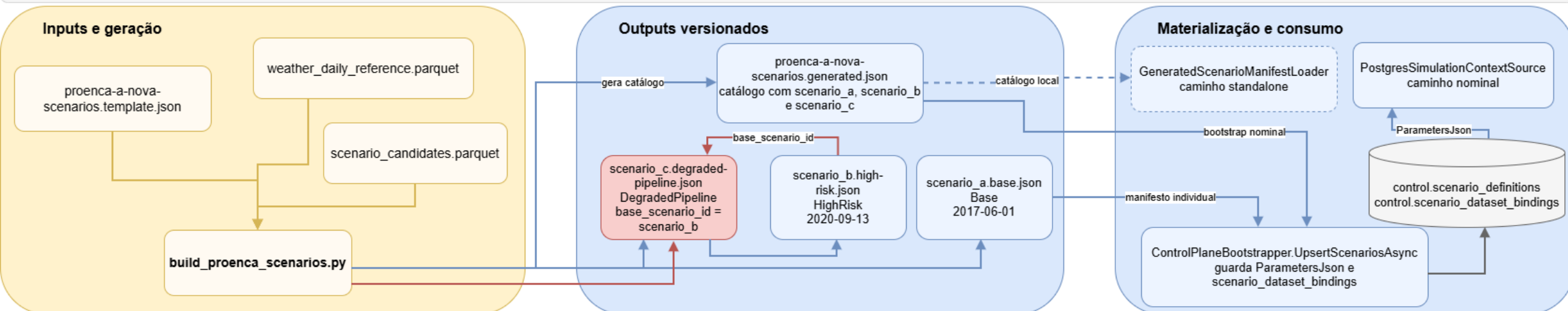
Da origem externa ao estado operacional: dados preparados e harmonizados alimentam cenários, os cenários alimentam o simulador, e os eventos alimentam o fluxo de prevenção.



Este é o fluxo completo desde dados preparados e cenários até execução operacional. A ideia é mostrar que a demo não começa só no código, começa na preparação dos dados e termina em API, dashboards e evidência.

Como os cenários são gerados, materializados e consumidos

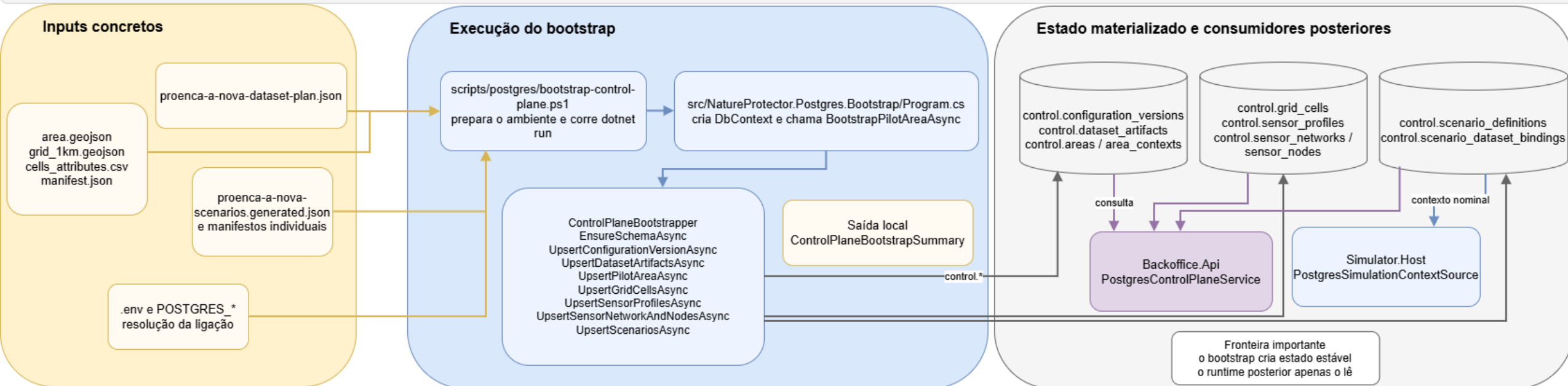
A vista mostra o template, os inputs curados, a geração dos cenários A/B/C, a materialização em control.scenario_definitions e o caminho alternativo standalone por GeneratedScenarioManifestLoader.



Este diagrama mostra como os cenários são gerados, versionados e consumidos. Isto liga diretamente à pesquisa, porque os cenários A, B e C não são apenas exemplos soltos; são artefactos que a runtime consome.

Bootstrap e materialização do control plane em PostgreSQL

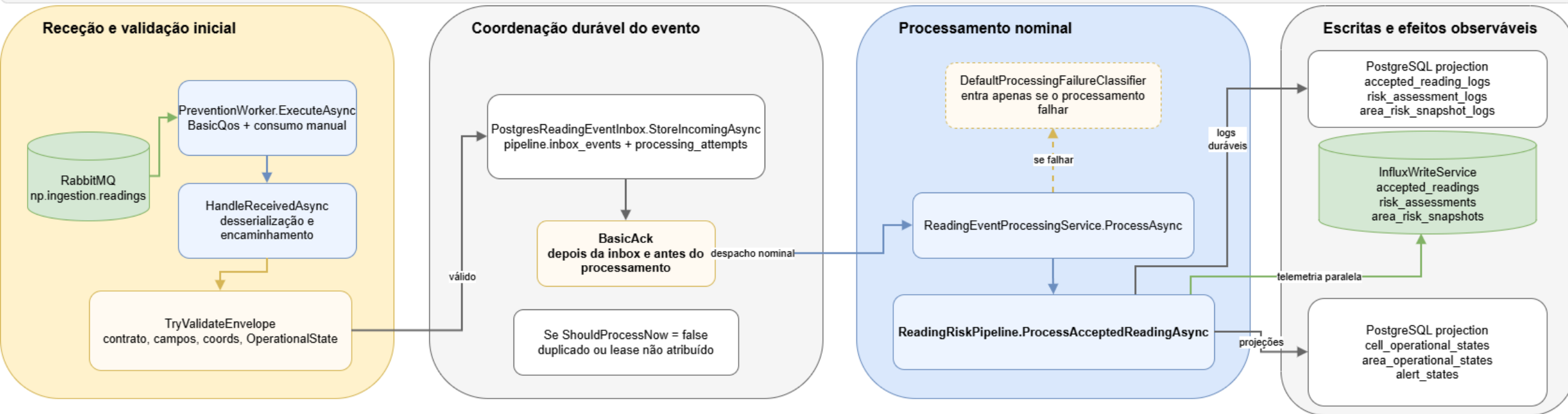
O bootstrap lê baseline e manifestos, aplica migrations e materializa um estado estável em control.*. O runtime posterior não volta a construir esse estado: apenas o consome.



Aqui mostramos como o plano de controlo é materializado em PostgreSQL: área, grelha, sensores e cenários deixam de estar apenas em ficheiros e passam a ser estado consultável pelo sistema.

Fluxo nominal de prevenção na runtime atual

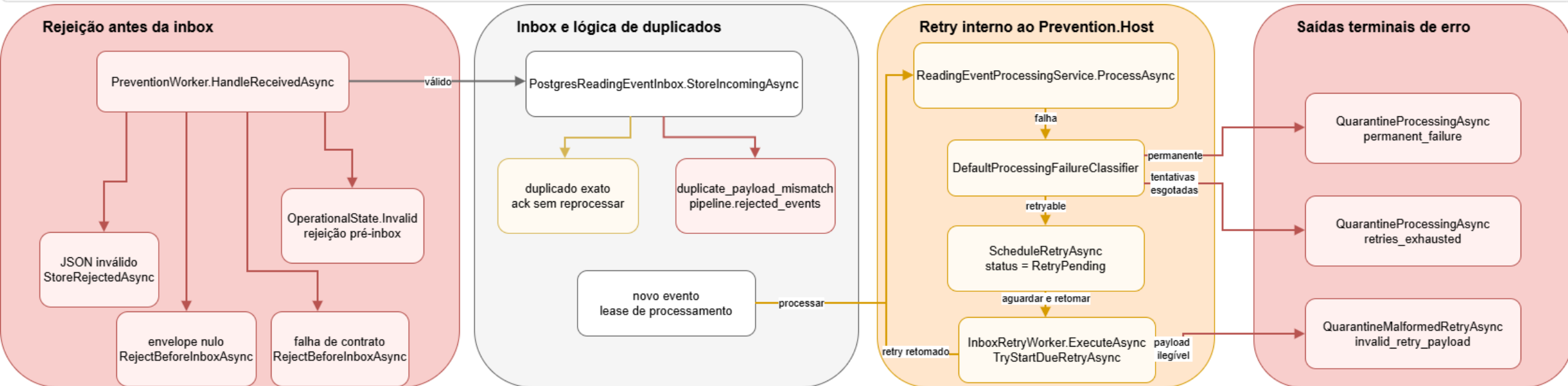
A vista separa recepção, validação, inbox, ack, processamento, projeções e telemetria. DefaultProcessingFailureClassifier aparece apenas no caminho de falha e não faz parte do caminho nominal.



Este é o caminho típico: o evento chega, é validado, passa pela inbox, é processado, gera avaliação de risco e produz efeitos observáveis em PostgreSQL, InfluxDB e logs.

Rejeição, retry e quarentena na pipeline atual

A vista separa rejeição antes da inbox, duplicados, retry agendado, retry retomado e quarentena. Todo o retry é interno ao Prevention.Host e não regressa ao broker.



Este slide mostra que não tratamos só o caso ideal. Eventos rejeitados, duplicados, atrasados ou com falha podem ser rejeitados, repetidos ou enviados para quarentena, mantendo rastreabilidade.

O que falta e trabalho futuro

Modelo de risco

- Substituir o scoring atual por um modelo mais fundamentado
- Integrar índices reais como FWI / KBDI
- Melhorar explicabilidade dos resultados

Progresso realizado: 60%

Dados e simulação

- Ligar datasets reais à configuração dos cenários
- Tornar a simulação mais realista: sensores, falhas, ruído, tempo
- Expandir para mais áreas, mais sensores e cenários mais longos

Por realizar: 40%

Estimativa baseada na baseline implementada, requisitos abordados e trabalho restante para validação final.

Backoffice e validação final

- Completar API, dashboards e visualização operacional
- Validar o sistema com a área piloto
- Fechar uma demonstração reprodutível e pronta para apresentação



DEMO



NatureProtector

Juntas-te a nós nesta missão de erradicar incêndios florestais?



SLIDES EXTRA

Reconhecimento

Ideia reconhecida com o 3º lugar num concurso de ideação de startups em Salzburg Austria.



Como se gere atualmente

Vigilância Humana (Torres de Vigia, Patrulhas)

- ✓ Forte Julgamento Local e Descoberta Rápida em áreas de alta visibilidade
- ✗ Limitado por fadiga, noite, fumo/nevoeiro e lacunas de visibilidade

Monitorização com câmaras (Torres fixas + Salas de Controlo)

- ✓ Cobertura 24/7 + confirmação visual e replay
- ✗ Requer eletricidade, manutenção e visibilidade + pode causar falsos alarmes

Sensores Remotos (Satélites + Informação Meteorológica/ de Fogos)

- ✓ Conhecimento em ampla área + útil para rastrear eventos
- ✗ Limites de Resolução/atrasos e menos sensível a ignições pequenas/recentes



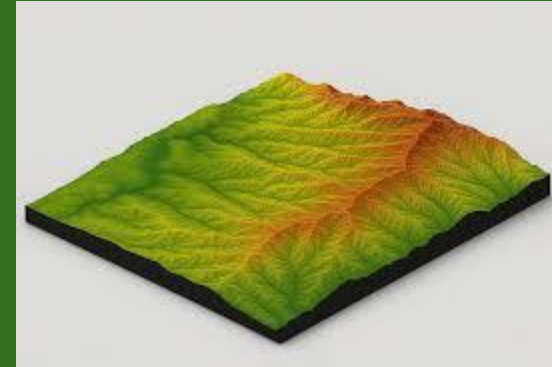
Lacunas nos sistemas existentes

- Cobertura territorial não homogénea
- Coordenação e propriedade fragmentadas
- Pontos cegos na cobertura
- Detecção tardia e confirmação lenta
- Alta dependência humana (carga de monitorização)



Simulação (pesquisado)

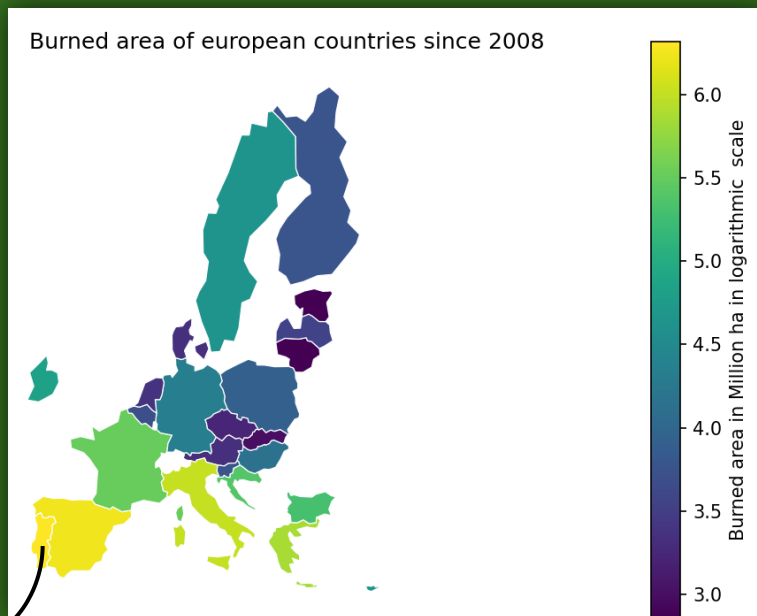
Digital Elevation Model (DEM) / Mapa



WindNinja - Simulação de Ventos



Dados e Gráficos

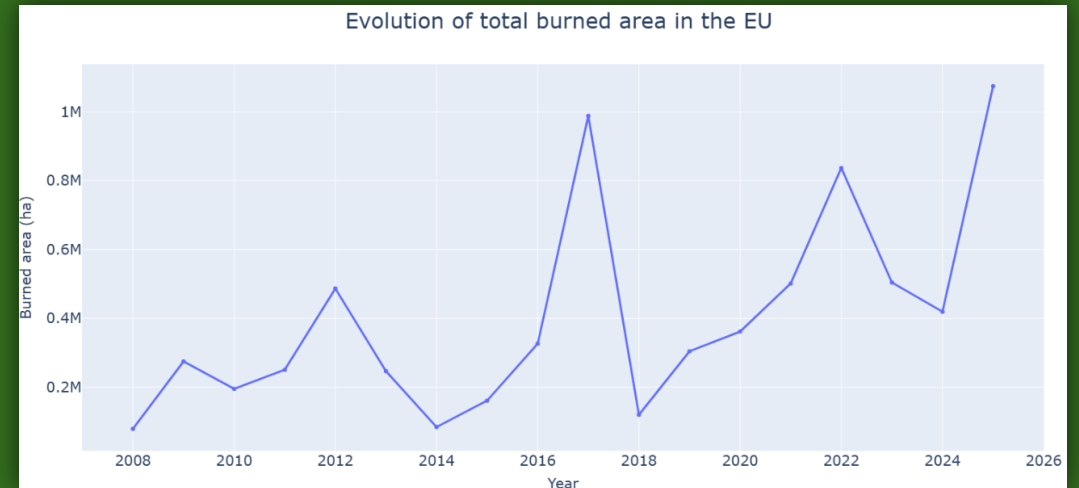


Países do Sul Europeu

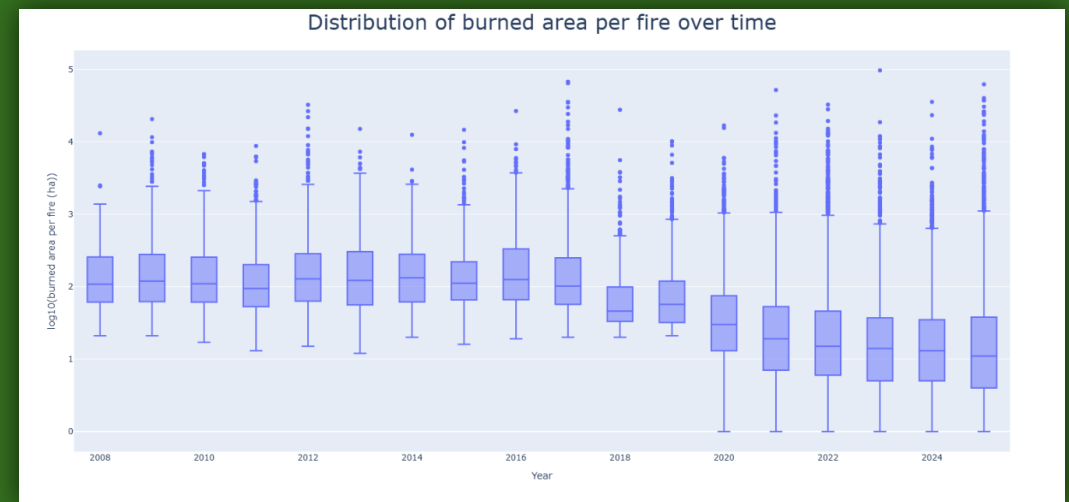


Desde 2008, ~¼ ardeu!!!

Dados originados de Copernicus/EFFIS
Gráficos feitos por Miguel



Tendência Crescente

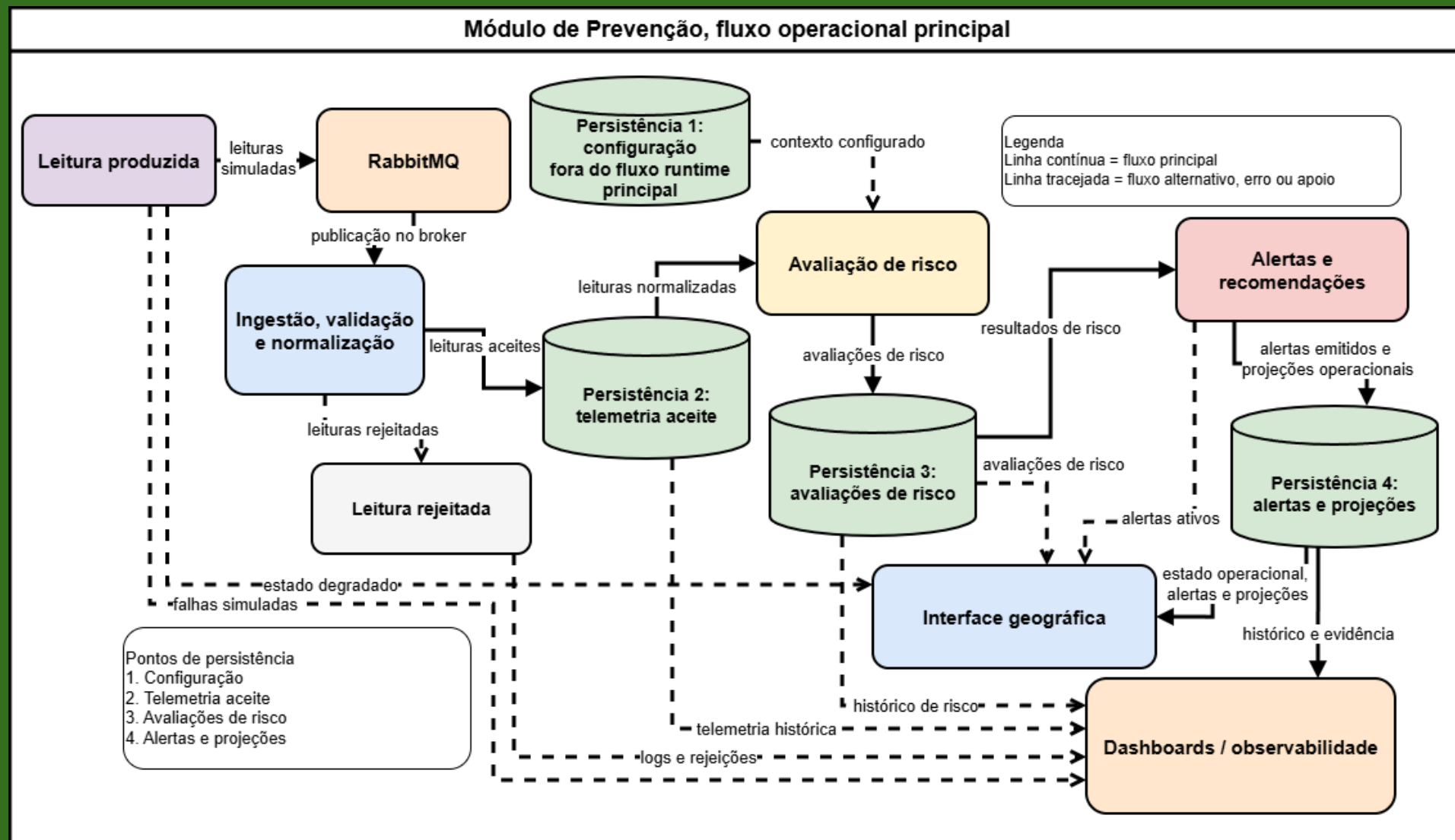


Imprevisibilidade

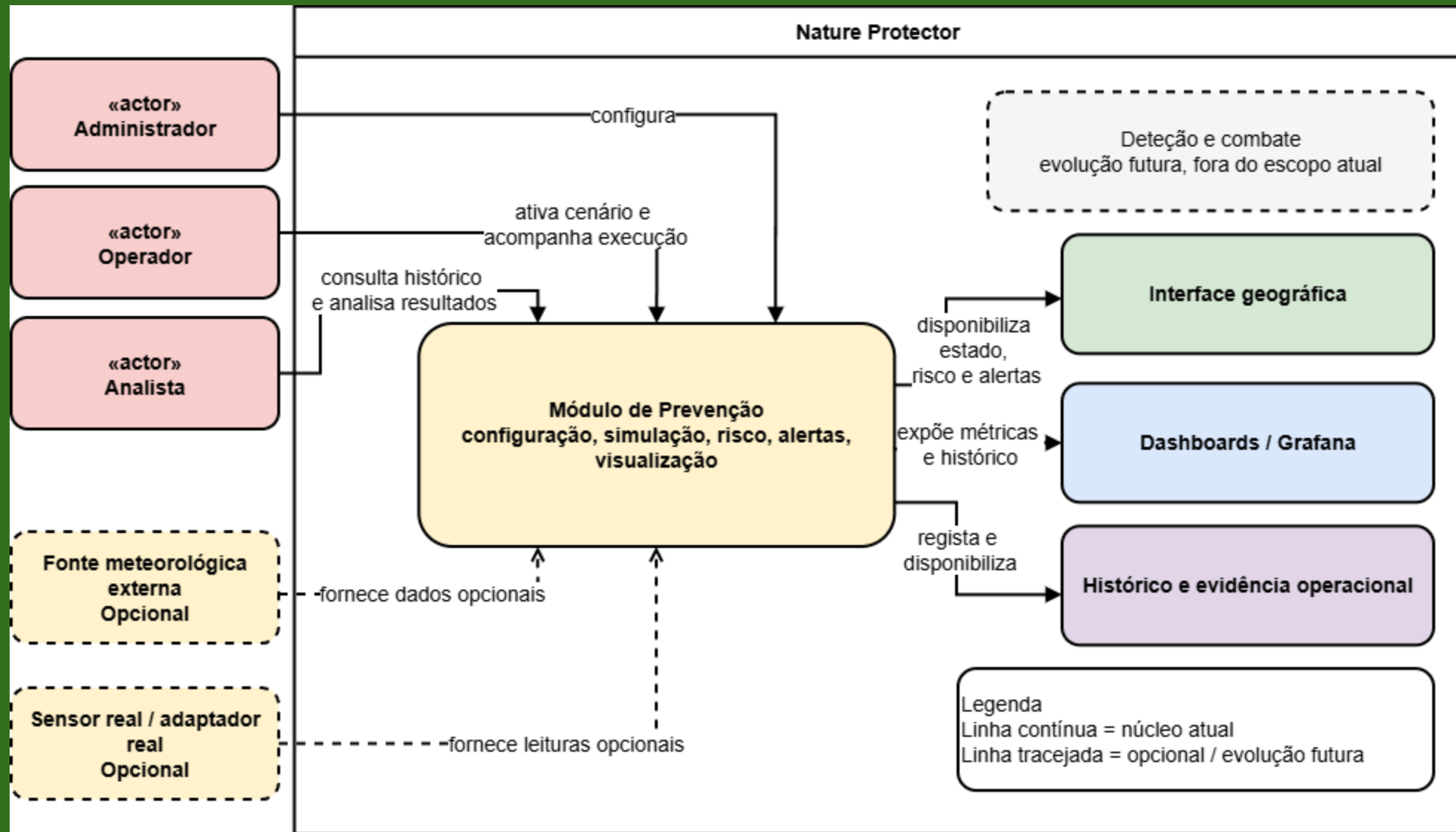
Solução

Pipeline

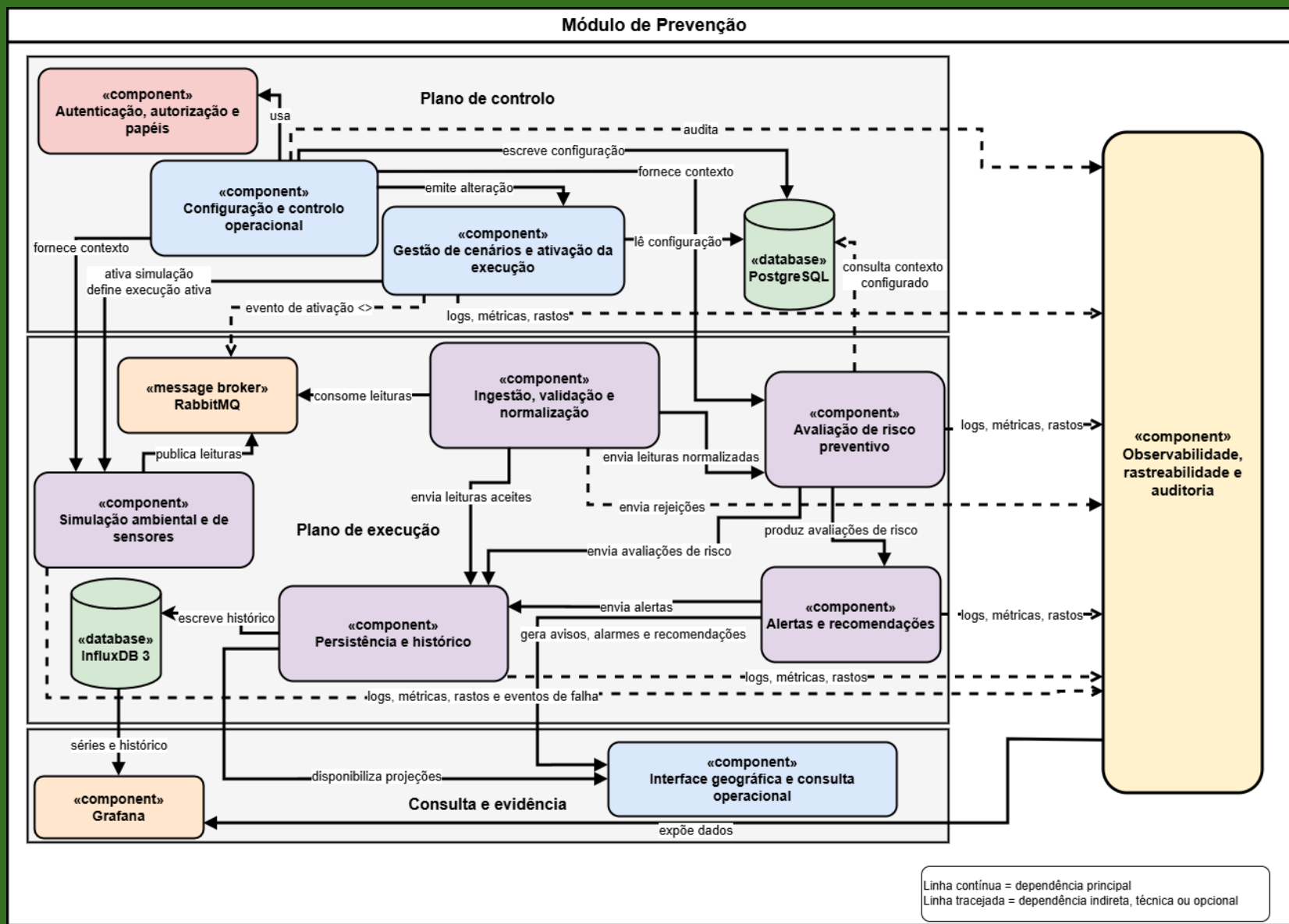
- Sensores simulados
- RabbitMQ
- Ingestão e validação
- Telemetria aceite
- Avaliação de risco
- Alertas e recomendações
- Interface geográfica
- Dashboards e evidência



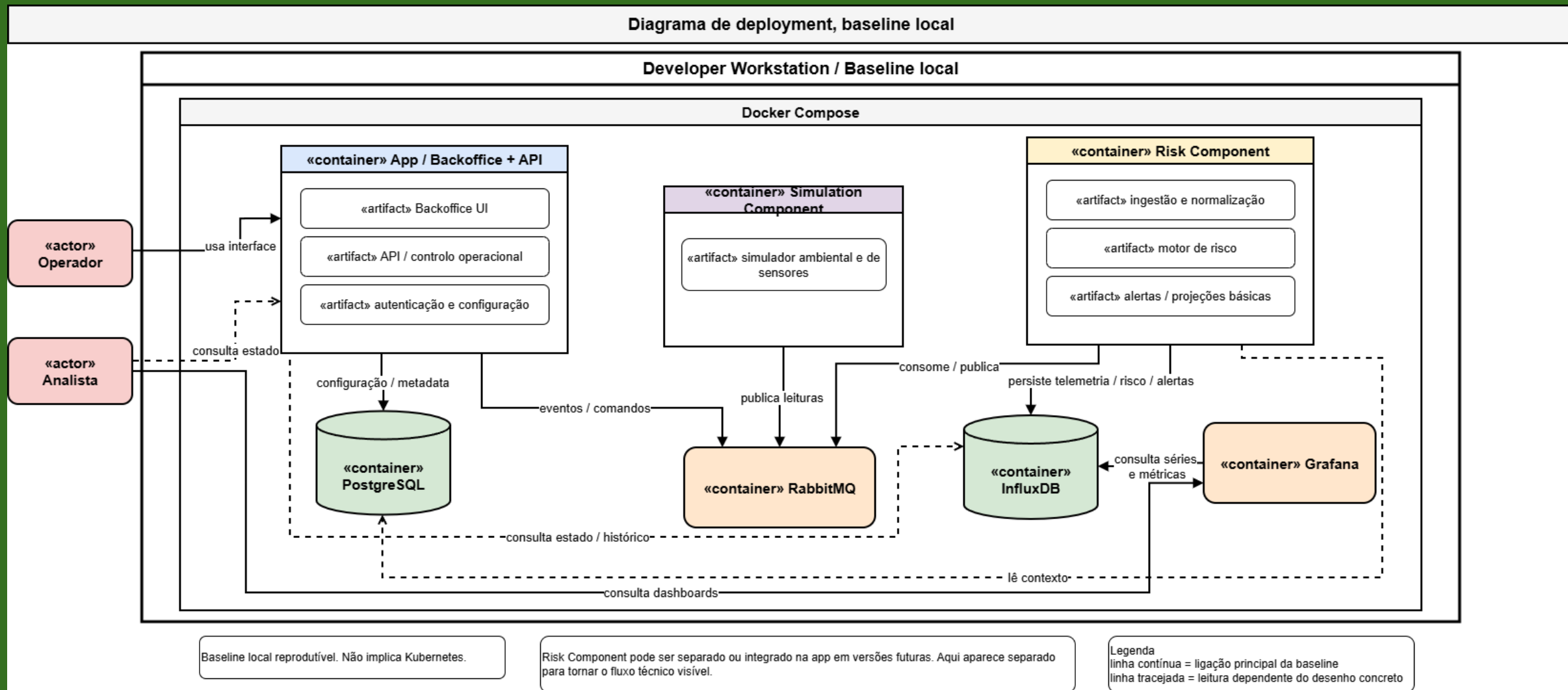
Contexto



Componentes lógicos



Deployment



Arquitetura em camadas do simulador

Estado atual implementado
SimulationRunner + ReadingGenerationService já cobrem 1, 2 e parte de 3.
A camada 4 ainda é sobretudo alvo.

Inputs

Manifest do cenário
A, B ou C

Plano de controlo
areas, sensors, profiles, scenarios

Seed determinística
e relógio lógico

Camadas arquiteturais do simulador

1. Resolução de contexto
ScenarioContext, área, sensores, perfis, run metadata

2. Camada de verdade física ou quase física
base temperature, humidity, wind, passo temporal e contexto do cenário

3. Camada de observação do sensor
ruído, disponibilidade, failure rate, estado Invalid

4. Camada de transporte e degradação
atraso, duplicação, out-of-order, burst outage

5. Envelope e publicação
EventEnvelope + SensorReadingProduced + routing key

Outputs

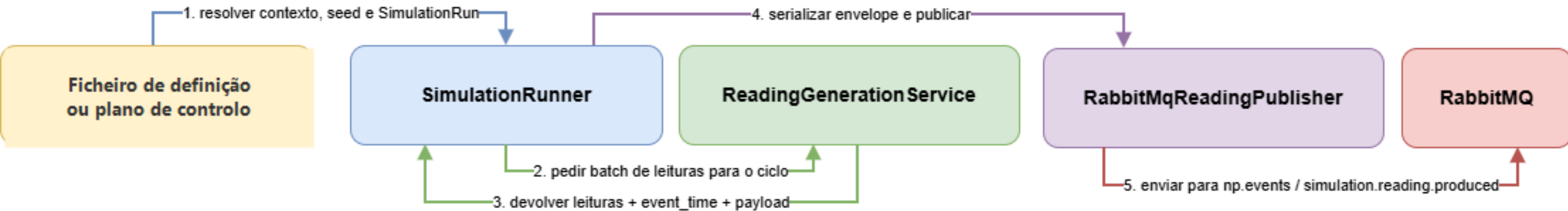
metadata da run

SimulationRun
event_time, ingest_time,
correlation_id

RabbitMQ
np.events /
simulation.reading.produced

publish

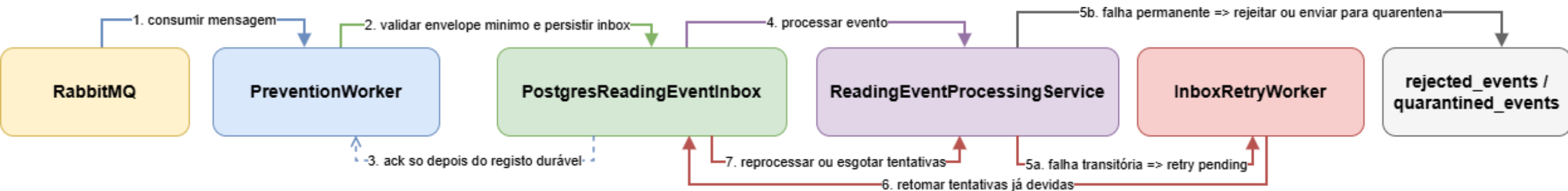
Sequencia nominal da simulação



Detalhes observáveis do caminho nominal

- o contexto pode vir de um ficheiro de definição ou do plano de controlo
- o relógio lógico deriva de $\text{StartTimestamp} + \text{ciclo} * \text{intervalo}$
- cada leitura sai em EventEnvelope com correlation_id e event_time
- o publisher efetivo na runtime atual é RabbitMqReadingPublisher

Nova tentativa e quarentena no fluxo operacional



Estados persistidos

pipeline.event_inbox
payload bruto, estado de processamento,
first_seen

pipeline.processing_attempts
attempt number, stage, result, error

pipeline.rejected_events
falhas estruturais ou permanentes

pipeline.quarantined_events
falhas esgotadas ou não resolvidas

Persistência, contratos e vistas operacionais

Contrato comum

EventEnvelope
schema_version
event_id
correlation_id
producer
event_type
area_id
event_time
ingest_time
payload

PostgreSQL

control

areas, grid_cells, sensor_profiles,
sensor_nodes, scenario_definitions,
dataset_artifacts,
scenario_dataset_bindings,
simulation_runs

pipeline

event_inbox
processing_attempts
rejected_events
quarantined_events

projection

accepted_reading_log
risk_assessment_log
area_risk_snapshot_log
cell_operational_state
area_operational_state
alert_state

liga configuração, datasets e runs

payload e semantica temporal

materializa estado operacional

Semantica temporal

event_time = tempo do fenomeno
ingest_time = entrada na pipeline
processing attempts = tempo de decisão operacional

InfluxDB

accepted_readings

telemetria aceite

avaliações de risco

risk_assessments

snapshots de area

area_risk_snapshots

Decisões de design e justificação

- **Foco exclusivo no módulo de prevenção, para garantir controlo de escopo**
- **Execução pequena, arquitetura preparada para crescer**
(Demonstrar com 1-3 áreas, 5 – 20 sensores projetar para cobrir todas as áreas florestais do mundo)
- **Separação entre plano de controlo e plano de execução**
- **PostgreSQL como fonte de verdade da configuração**
- **InfluxDB para telemetria e histórico operacional**
- **RabbitMQ para desacoplamento entre simulação, ingestão, risco e alertas**
- **Grafana para dashboards**
- **Simulação como base da demonstração, para garantir repetibilidade e independência de hardware real**
- **Regras empíricas de risco em primeira instância e opcionalmente usando aprendizagem automática**

Riscos e mitigação

Riscos:

- **Simulação pouco credível**
- **Regras empíricas fracas e falsos alarmes**
- **Complexidade excessiva da arquitetura**
- **Dependência de integrações opcionais**
- **Falta de alinhamento com utilizadores reais**

Mitigação:

- **Pesquisa e calibração por referências técnicas, perfis de comportamento e cenários comparáveis**
- **Validação incremental, comparação de hipóteses, níveis de severidade e afinação por histórico**
- **Foco no núcleo de prevenção e requisitos fundamentais, mantendo extensões como opcionais**
- **Modo simulado, adaptação por interfaces e evidência incorporada (métricas e registos correlacionados)**
- **Entrevistas e consulta a utilizadores do domínio**

Porquê estas tecnologias?

A prioridade não foi maximizar sofisticação tecnológica; foi garantir uma baseline técnica estável, demonstrável e preparada para crescer.

PostgreSQL

Fonte de verdade da configuração, regras, cenários, utilizadores e auditoria.

Escolhido porque o plano de controlo tem entidades estáveis e relacionais.

InfluxDB

Armazena telemetria, histórico operacional, avaliações de risco, alertas e métricas observáveis.

Escolhido porque o plano de execução produz dados temporais e históricos de consulta.

RabbitMQ

Desacopla simulação, ingestão, avaliação de risco, persistência e alertas.

Escolhido para suportar pipeline por eventos sem colar produtores e consumidores.

Grafana

Suporta dashboards, métricas, lag, throughput, latência e evidência operacional.

Escolhido porque observabilidade e demonstração técnica são requisitos da fase atual.

Docker Compose

Garante baseline local executável e reproduzível.

Escolhido para demo estável, local e com dependências controladas.

Simulação dos Sensores e Regras Empíricas

- **Simulação orientada a cenários:** cada sensor parte de uma baseline ambiental por área e por instante.
- **Evolução temporal suave:** uso de componente sinusoidal para representar ciclos e variações graduais.
- **Erro e ruído de medição:** uso de distribuições estatísticas para aproximar comportamento real.
- **Distribuição sinusoidal:** modela tendência temporal e flutuação periódica.
- **Distribuição de Rayleigh:** modela erro positivo e variações associadas a intensidade e dispersão.
- **Possibilidade de combinar com ruído adicional e falhas ocasionais do sensor.**
- **Regras empíricas de risco:** score calculado a partir de temperatura, humidade, vento e contexto da área.
- **Abordagem explicável:** cada fator contribui com pesos ajustáveis e thresholds observáveis.
- **Objetivo:** gerar dados plausíveis, reproduzíveis e úteis para testar ingestão, persistência e assessment.

Requisitos Funcionais

- **Configurar áreas e respectivas grelha de risco**
- **Definir rede de sensores simulados por área**
- **Executar cenários preparados previamente**
- **Gerar leituras ambientais plausíveis, (temperatura, humidade e vento)**
- **Calcular risco preventivo por célula e por área**
- **Emitir avisos e alertas com base no risco calculado**
- **Disponibilizar visualização geográfica, histórico e evidência operacional**

Requisitos Não Funcionais

- **Reprodutibilidade**, os mesmos cenários devem poder ser repetidos com resultados semelhantes
- **Rastreabilidade**, deve ser possível seguir o fluxo entre leitura, risco e alerta
- **Observabilidade**, o sistema deve expor métricas, logs e histórico
- **Modularidade**, a arquitetura deve permitir evolução futura sem reestruturação total
- **Auditabilidade**, alterações de configuração devem ser identificáveis
- **Controlo de âmbito**, a solução deve caber no semestre e ser demonstrável
- **Extensibilidade**, preparar evolução futura para novas fontes e novos módulos